

Workshop

What 's new in EF Core 8



SoftUni Team
Technical Trainers



SoftUni



Software University

<https://softuni.bg>

sli.do

#csharp-db

Table of Contents

1. New Features in EF Core 8
2. Model Creation
3. Queries
4. Concurrency
5. Transactions
6. Performance
7. Interceptors
8. Nullable References Types





Complex Types

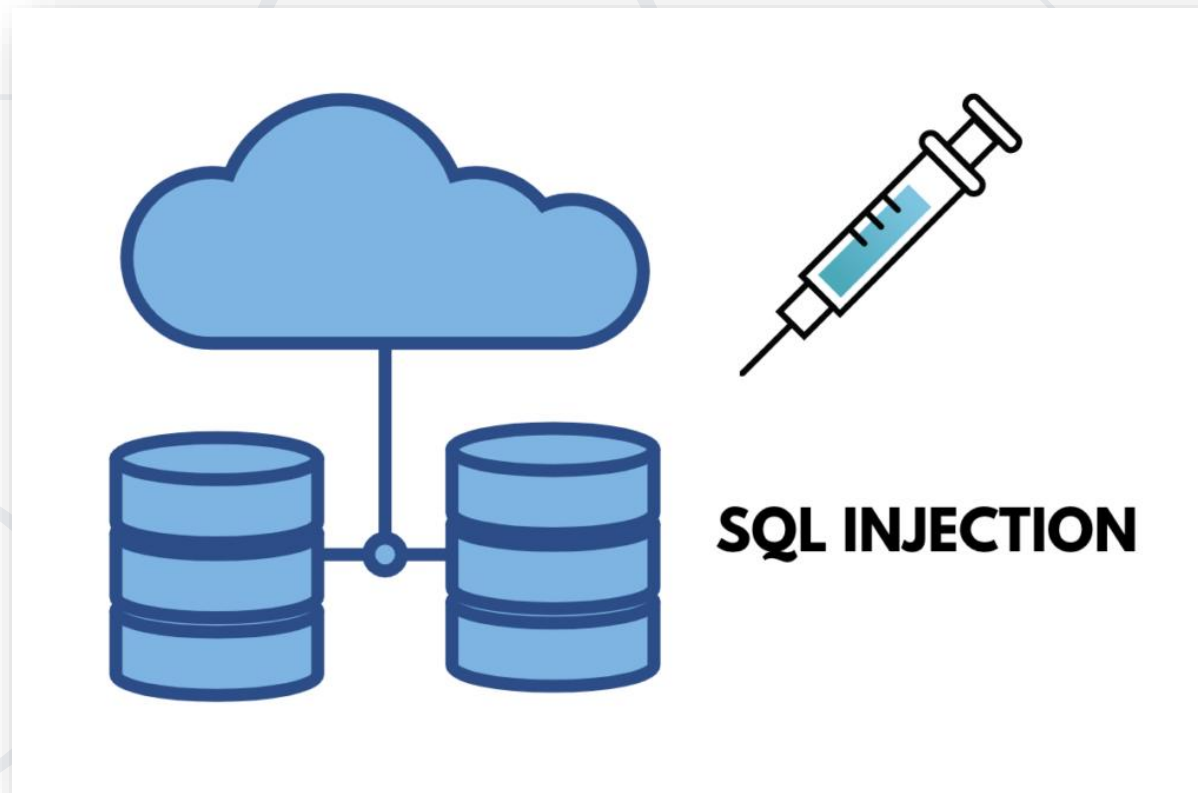
- Previously:
 - Only **primitive types** (columns) and **entity types** (tables)
- Complex types
 - Do not have a **Primary Key**
 - Must be **part of an entity** (i.e. cannot be a DbSet)
 - Can be **.NET value or reference types**
 - **Multiple properties** can share **one instance** (not possible with Owned types)
 - Must be marked with **[ComplexType] attribute**

- **PostgreSQL** is among the few RDBMSs with **built-in support** for collections of **primitive types**
- **EF Core 8** adds support (not available before)
- For databases **lacking native support**
 - **JSON serialization** is used
- Data is **stored in supported column types** (e.g., **string**)
- **SQL Server**: min. supported version is **SQL Server 2016** (compatibility level 130)
- **PostgreSQL**: best used with **ICollection<T>**

- Storing **serialized data** is becoming more common
- Used in **document databases** (e.g., **MongoDB**, **Cosmos DB**) or **RDBMS**
- **PostgreSQL**: uses **jsonb** (JSON blob)
 - extended **SQL operators and functions**
- **SQL Server**: uses **varchar**, includes **JSON functions**
- **EF Core 8**: uses **Owned Type**
 - **ToJson()** extension method in **OnModelCreating**

- **EF 7** allowed scalar results
- **EF 8** expands with
 - **SQL Query<T>()**
 - **SqlQueryRaw<T>()**
- **Result types** can use **[Column]** for column-property mapping
- Returns **IQueryable**, usable with **LINQ**
- **EF 8 optimizes queries** if it handles the entire SQL

- Watch out for **SQL Injection!**



- **HasDefaultValue()** and **HasDefaultValueSql()**
- **Sentinel** Values
 - Before EF 8 Core - .Net default value
 - With EF 8 Core - **HasSentinel()**
- **Default Boolean:** EF 8 treats **default true as sentinel**
- **Default Enum:** same issues as Boolean now **better handled**

ExecuteUpdate & ExecuteDelete

- Introduced in **EF Core 7**
- Improved in **EF Core 8**
- Allow **bulk update/delete** directly
- New **bypass in EF workflow**
 - **Fetch/Track/Save**
 - **Update or Delete**
 - Do not mix both approaches
- **Not tracked** by Change Tracker -> **ExecuteUpdate / ExecuteDelete**
- Execute **immediately** (not part of **UnitOfWork pattern**)

- By design, they should **only affect a single table**
- EF Core 8 fixes an issue in EF Core 7 by **adding support for Owned Entities**
- In EF Core 7:
 - **Exception** would be thrown since it accesses **two entities**
 - In reality, they are stored in the **same table**

ExecuteUpdate & ExecuteDelete

```
public class Customer
{
    public int Id { get; set; }
    public required string Name
    public required CustomerInfo
```

[Owned]

```
public class CustomerInfo
{
    public string? Tag { get; set; }
}
```

```
await context.Customers
    .Where(e => e.Name == name)
    .ExecuteUpdateAsync(s =>
        s.SetProperty(b => b.CustomerInfo.Tag, "Tagged")
        .SetProperty(b => b.Name, b => b.Name + "_Tagged"));
```

```
UPDATE [c]
SET [c].[Name] = [c].[Name] + N'_Tagged',
    [c].[CustomerInfo_Tag] = N'Tagged'
FROM [Customers] AS [c]
WHERE [c].[Name] = @name
```

- **EF Core 7 and Older**

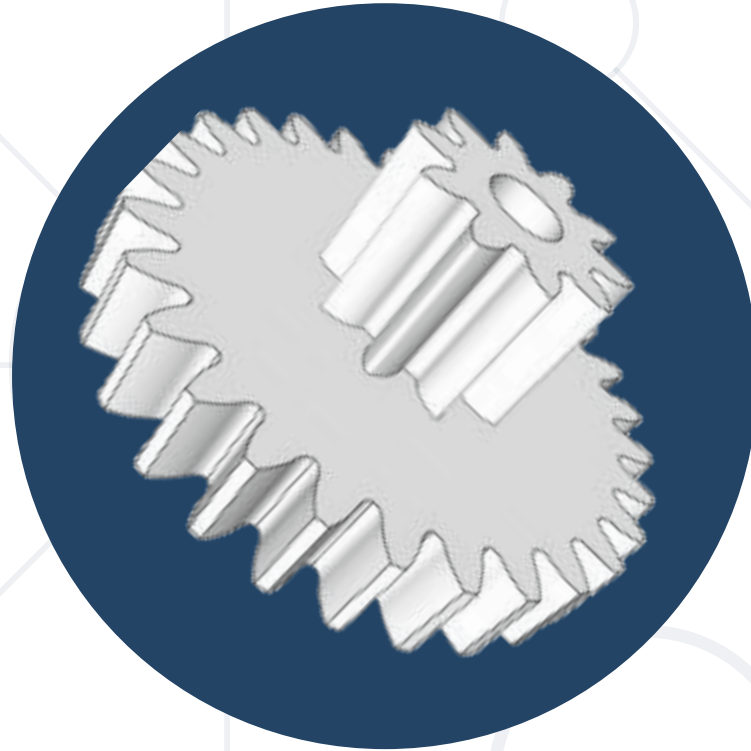
```
var blogsWithPosts = await context.Blogs
    .Where(b => context.Posts.Select(p => p.BlogId
    .ToListAsync();
```

```
SELECT b."Id", b."Name"
FROM "Blogs" AS b
WHERE EXISTS (
    SELECT 1
```

- **EF Core 8**

```
SELECT b."Id", b."Name"
FROM "Blogs" AS b
WHERE b."Id" IN (
    SELECT p."BlogId"
    FROM "Posts" AS p
)
```

```
    "Posts" AS p
    "BlogId" = b."Id"
```



Model Creating

- When creating the model, **Data Annotation** is preferred.
 - However, if **Fluent API** is needed
 - Separate **configuration files**
- Example:
 - **Exclude a table** from migrations
 - **Include a View** in the model
- Use the **[Comment]** attribute to add comments at the table level in the database

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    ...
    modelBuilder.Entity<IdentityUser>()
        .ToTable("AspNetUsers", t => t.ExcludeFromMigrations());
}
```

```
modelBuilder.Entity<Blog>()
    .ToView("blogsView", schema: "blogging");
```


- Always use **[Comment]** for every property
- Always set a **maximum length** for string properties (unless **explicitly required** as **text** / **varchar(max)**).
 - Use **[MaxLength]** or **[StringLength]**
- Always use **[Required]** for properties that should **not** be **Nullable** (to avoid confusion with **NRT**)
- For **decimal** properties, it is recommended to use **[Precision]**
- The attribute **[Column(Order = n)]** can be used to **define the column order** when **creating the table**

- **Primary Key**, use `[Key]` at the **property level**
- **Composite Primary Key**, use `[PrimaryKey(nameof(first), nameof(second))]` at the **table level**
- For **non-composite numeric and GUID keys**
 - EF Core automatically generates values
 - If we do not want them to be auto-generated, use `[DatabaseGenerated(DatabaseGeneratedOption.None)]`
- The **HiLo strategy** can be used: `UseHiLo()`
- **Alternative keys** can also be defined

■ Default Values

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Blog>()
        .Property(b => b.Rating)
        .HasDefaultValue(3);
}
```

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Blog>()
        .Property(b => b.Created)
        .HasDefaultValueSql("getdate()");
}
```

■ Computed Columns

```
modelBuilder.Entity<Person>()
    .Property(p => p.DisplayName)
    .HasComputedColumnSql("[LastName] + [FirstName]", stored: true);
```

```
modelBuilder.Entity<Person>()
    .Property(p => p.NameLength)
    .HasComputedColumnSql("LEN([LastName]) + LEN([FirstName])", stored: true);
```

- By **convention**, indexes are added to **Foreign Key properties** (can be disabled)
- At the table level, the **[Index] attribute** can be used
 - **Single-column index**: **[Index(nameof(prop))]**
 - **Composite index**: **[Index(nameof(first), nameof(second))]**
 - **Unique index**: **[Index(nameof(prop), IsUnique = true)]**

- Index with **additional included columns** that are not part of the key

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Post>()
        .HasIndex(p => p.Url)
        .IncludeProperties(p => new { p.Title, p.PublishedOn });
}
```

- Check constraints

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Product>()
        .ToTable(b => b.HasCheckConstraint("CK_Prices", "[Price] > [DiscountedPrice]"));
}
```

- Prefer **avoiding inheritance**
- Three strategies:
 - TPH (**Table Per Hierarchy**) – default strategy
 - TPT (**Table Per Type**) – use `.UseTptMappingStrategy()`
 - TPC (**Table Per Concrete Type**) – uses denormalized data and a shared sequence



- Requires extra packages:

- SQL Server

Microsoft.EntityFrameworkCore.SqlServer.NetTopologySuite

- PostgreSQL

Npgsql.EntityFrameworkCore.PostgreSQL.NetTopologySuite

```
options.UseSqlServer(  
    @"Data Source=(localdb)\MSSQLLocalDB;Initial Catalog=WideWorldImporters",  
    x => x.UseNetTopologySuite());
```

- **Supported types:** Geometry, Point, LineString, Polygon, etc.
- $X = \text{Longitude}$, $Y = \text{Latitude}$ (note the **reversed order**)
- **PostgreSQL:** requires **postgis** extension

```
protected override void OnModelCreating(ModelBuilder builder)
{
    builder.HasPostgresExtension("postgis");
}
```


- [Spatial Data in the SQL Server Provider](#)
- [Spatial Data in the Npgsql Provider](#)
- [NetTopologySuite Docs](#)

- We can add a **sequence** to the model, for example, for use as **part of a numbering system**

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.HasSequence<int>("OrderNumbers", schema: "shared")
        .StartsAt(1000)
        .IncrementsBy(5);
}
```

- **Value Converters** – allow **converting values** when reading and writing to the database

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
```

```
{
```

```
    modelBuilder
```

```
        .Property
```

```
{
```

```
        .HasConversion
```

```
        v
```

```
        v
```

```
}
```

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
```

```
    modelBuilder.Entity<Rider>()
```

```
        .Property(e => e.Mount)
```

```
        .HasConversion<string>();
```

```
}
```

- Value converters can be applied to all properties of a given type in

```
protected override void ConfigureConventions(ModelConfigurationBuilder configurationBuilder)
{
    configurationBuilder
        .Properties<Currency>()
        .HaveConversion<CurrencyConverter>();
}

public class
{
    { }
}
```

- Predefined Converter

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder
        .Entity<Rider>()
        .Property(e => e.Mount)
        .HasConversion<string>();
}
```

■ Initial Data Seeding

```
modelBuilder.Entity<Blog>().HasData(new Blog { BlogId = 3, Url = "http://sample.com" });
```

- **Primary keys**, even though they are usually generated by the database, **must be explicitly provided**. They are used to track changes between migrations
- Seeded records **will be removed in the next migration** if their **primary keys** change

- **Table Splitting:**
 - **Multiple Entity classes** mapped to a **single table**
 - Sometimes we **want to preserve** object-oriented composition, but it **doesn't align well** with the relational model
- **Entity Splitting:**
 - **Multiple tables** mapped to a **single Entity model** (not recommended)



Queries

- Client vs Server evaluation
- **Tracking** vs. **No-Tracking** Queries
- Loading Related Data
 - **Eager** , **Explicit** , **Lazy**

```
using (var context = new BloggingContext())
{
    var blogs = context.Blogs
        .Include(b => b.Posts)
        .Include(b => b.Owner)
        .ToList();
}
```

```
using (var context = new BloggingContext())
{
    var blog = context.Blogs
        .Single(b => b.BlogId == 1);

    context.Entry(blog)
        .Collection(b => b.Posts)
        .Load();

    context.Entry(blog)
        .Reference(b => b.Owner)
        .Load();
}
```


- **Avoid serializing Entity types.** Instead, use **DTO** / **ViewModel**

```
services.AddControllers()  
    .AddJsonOptions(options =>  
    {  
        options.JsonSerializerOptions.ReferenceHandler = ReferenceHandler.IgnoreCycles;  
    });
```

- EF uses the **JOIN** operator to load related data
- Although **JOIN** is a standard operation in **RDBMS**, it can lead to performance issues
- **Cartesian Explosion**
 - Happens when navigation properties are included at the same level
 - If a **blog** has **10 posts** and **10 authors**, **100 rows** will be returned, which can cause **serious performance issues**.

```
var blogs = ctx.Blogs
    .Include(b => b.Posts)
    .Include(b => b.Contributors)
    .ToList();
```

- **Data Duplication**

- In the **example above**, each of the **100 rows** will return **all columns** from the blog
- If the blog contains **large columns** (e.g., file contents), this **further impacts performance**

- **Solution**

- **Both problems** can be solved using **Split Query**
- **There is no strict rule** on when to use **Single Query** vs. **Split Query**

- By default, EF uses **Single Query**
- A query must be explicitly marked as a **Split Query**

```
using (var context = new BloggingContext())  
{  
    var blogs = context.Blogs  
        .Include(blog => blog.Posts)  
        .AsSplitQuery()  
        .ToList();  
}
```

- Databases typically guarantee **consistency** within a single query, but not across multiple queries
 - With **Split Query**, there is a risk of retrieving **outdated data** that has already been modified
- Each query is an additional network operation, which can be a problem when network **latency** is high
- Some databases support **batching multiple queries**, but most do not
 - This means the result must be **cached** in memory, which can increase **memory consumption** in the application
- When using **Split Query** with **pagination** (**Skip / Take**), ensure that **ordering guarantees uniqueness**

- **LINQ GroupBy**

- The result of the **LINQ GroupBy** operator is **IGrouping<TKey, TElement>**, where **IGrouping** implements the **IEnumerable<TElement>** interface
- This allows **further processing** with LINQ

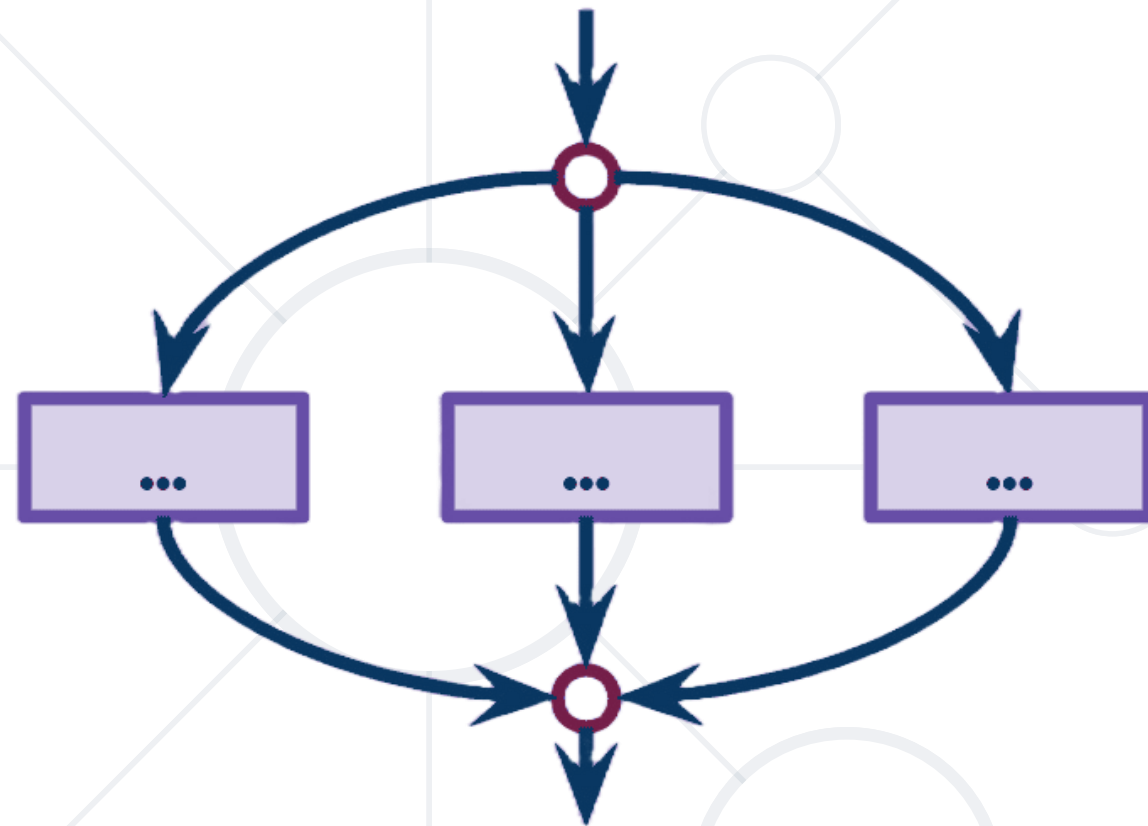
- Databases do not have a concept of **IGrouping**, which is why **most LINQ GroupBy queries cannot** be translated to SQL
- The **SQL GROUP BY operator** is quite **restrictive**
 - You can **group only by scalar values**
 - The **projection** can include only **columns** that are **either part of the key** or **aggregated**
- **EF analyzes the query** and translates it to SQL **if** it meets these conditions
- In some cases, starting from **EF Core 7**, grouping may be applied **after the query execution**

- **Global filters** are predicates added to the **model metadata** and applied to **every query**
- They **also apply** when data is retrieved through **navigation properties**
- Most used for
 - **Soft Delete** (isActive)
 - **Multi-tenancy** (tenantId)

```
modelBuilder.Entity<Blog>().HasQueryFilter(b => EF.Property<string>(b, "tenantId") == tenantId);  
modelBuilder.Entity<Post>().HasQueryFilter(p => !p.IsDeleted);
```


- Using Required Navigation properties with **an active global filter** may cause the **main entity** to be **excluded** from the result (**translates to INNER JOIN**)
- Disabling global filters for a specific query
 - This **removes all global filters** applied to the selected entity

```
blogs = db.Blogs
    .Include(b => b.Posts)
    .IgnoreQueryFilters()
    .ToList();
```



Concurrency

- Two Strategies for Conflict Management
 - **Optimistic**
 - Assumes that conflicts are **rare**
 - **Decides what to do** after a conflict occurs
 - **Default strategy** in EF
 - **Pessimistic**
 - **Locks the data** before processing it
 - Ensures that **data is always up to date**
 - **No conflicts occur**, but **concurrent queries are blocked**

- EF Core implements the **optimistic strategy** by configuring a property as a **concurrency token**
- This property is loaded and tracked like any other, and on **update**, its value is compared to the database value

- To define properties for **automatically generated** database concurrency tokens, EF uses the **[Timestamp]** attribute
- The use of this attribute is **database-specific**
 - MS SQL Server
 - PostgreSQL

```
[Timestamp]  
public byte[] Version { get; set; }
```

- **MS SQL Server** uses a special type **rowversion**, where the database generates a value **on every Insert or Update**
- **PostgreSQL** has a similar type, but **another approach** can be used
 - Every table in PostgreSQL contains **additional system columns**, such as **xmin**, which holds the identifier of the last transaction that modified the row
- **For EF Core versions before 7**, use

```
modelBuilder.Entity<Blog>().UseXminAsConcurrencyToken();
```

- Allows tokens to be used in databases that do not support automatic generation (e.g., **SQLite**)
- Gives control over when we want to activate the mechanism
 - In some cases, conflict tracking is unnecessary
- To define properties for application-managed concurrency tokens, EF uses the **[ConcurrencyCheck]** attribute

```
[ConcurrencyCheck]  
public Guid Version { get; set; }
```

- On Object Modification

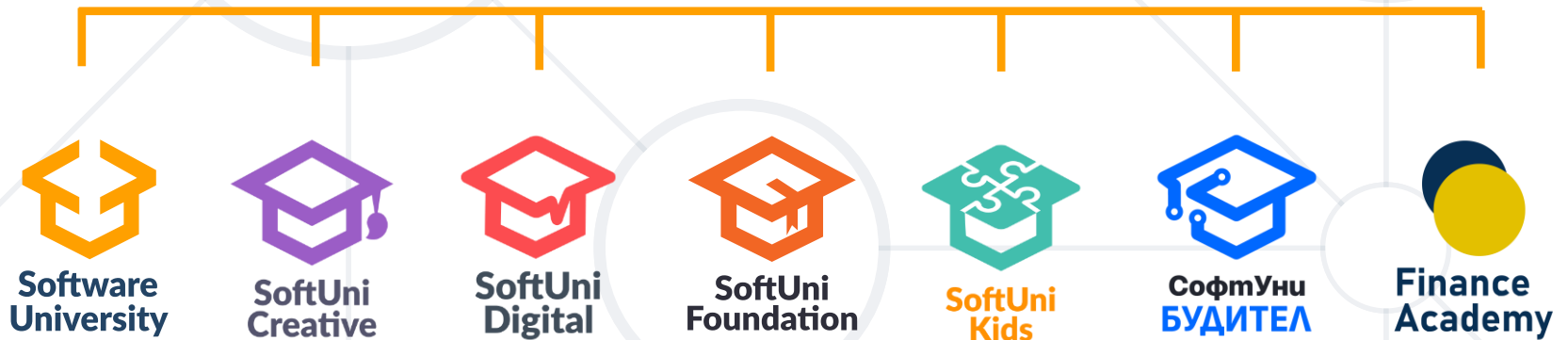
```
var person = context.People.Single(p => p.FirstName == "John");  
person.Age += 1;  
context.SaveChanges();
```

- If you always want a **new GUID** to be generated, you can use an **interceptor**
- However, it is generally better to have control over when the token is generated to avoid unnecessary conflicts

Questions?



SoftUni



SoftUni Diamond Partners



THE CROWN IS YOURS



- Software University – High-Quality Education, Profession and Job for Software Developers

- softuni.bg, about.softuni.bg

- Software University Foundation

- softuni.foundation

- Software University @ Facebook

- facebook.com/SoftwareUniversity



- This course (slides, examples, demos, exercises, homework, documents, videos and other assets) is **copyrighted content**
- Unauthorized copy, reproduction or use is illegal
- © SoftUni – <https://about.softuni.bg>
- © Software University – <https://softuni.bg>

